Informe técnico · TPI Food Store · Segunda entrega

Estudiante: Santiago Barretto

Repositorio: DB_Santiago_Barretto

Alcance: Unidades 4 a 8 · PostgreSQL 18, MongoDB y SQLite

Rama de esta integración: codex/tpi-segunda-entrega

Elementos implementados y decisiones

Unidad 4. Se verificó ResponsableControlID -> DepositoID, se descompuso control_lote_almacen en dos relaciones FNBC y se creó una vista de compatibilidad. La tabla original se conserva para reversión. La revisión final agregó guardas de DF y equivalencia antes del COMMIT: un caso válido confirmó la transacción y dos casos inválidos terminaron con código 3, sin confirmar cambios. Para el top de categorías se escogió una vista materializada derivada y refrescada por triggers. El ensayo de reversión desmontó las tablas derivadas dentro de una transacción y terminó con ROLLBACK, conservando las tres filas del origen. La spec de revisión fue redactada con Codex.

Unidad 5. Se separaron roles de lectura, escritura, soporte, reportes, auditoría y administración, con permisos por objeto y columna. Se habilitó la auditoría nativa en laboratorio y se simuló un usuario de soporte con autenticaciones fallidas y lectura masiva. Se corrigió el permiso sobre auditoria_evento: admin_datos ahora sólo tiene SELECT. La contención propuesta es preservar el log y suspender el login ante evidencia de compromiso; no se suspendió un usuario de laboratorio legítimo.

Unidad 6. Se prepararon chequeos de salud, ANALYZE condicional, respaldo custom con SHA-256 y manifiesto, y restauración en una base distinta. Se evita VACUUM FULL como rutina y no se almacenan credenciales en Git. En esta integración se registró una tarea diaria de Windows a las 20:00, se comprobó un disparo por horario con resultado 0 y se probó una falla simulada del respaldo. El chequeo y el respaldo son diarios, la revisión de mantenimiento los lunes y la limpieza queda a cargo del autovacuum. El logon Interactive exige sesión de Windows iniciada.

Unidad 7. Se modelaron pedidos con líneas embebidas en MongoDB y se compararon con PostgreSQL. Se mantiene PostgreSQL como origen autorizado; MongoDB puede servir lecturas de pedidos completos, pero no se midió latencia. El origen no tiene pedido.estado, por lo que se documentó explícitamente sin_estado_en_origen en lugar de inventar un estado.

Unidad 8. Se creó una base SQLite STRICT para catálogo, borradores, ítems y cola de sincronización. Un semáforo compara instantáneas con la copia PostgreSQL y clasifica SIMULADO_APROBADO, CONFLICTO o RECHAZADO. No confirma ventas ni escribe en PostgreSQL; un conflicto de precio se resolvió creando una solicitud nueva con otro UUID tras aceptar el precio actualizado. La spec fue redactada con Codex. OpenCode Build generó una variante final del esquema y semáforo, adoptada tras descartar un índice redundante, eliminar BOM y probar equivalencia de ocho solicitudes. El índice aceptado se añadió al archivo SQLite entregado sin alterar sus filas.

Cómo se probó y resultados observados

Unidad	Prueba y resultado real
4	DF sin conflictos; EXCEPT bidireccional con 0 diferencias antes del COMMIT. Dos pruebas negativas abortaron con código 3. Reporte: 49,870 ms normalizado y 0,344 ms materializado; auditoría del refresco con 0 diferencias. Reversión ensayada: 3 filas antes/después, vista conservada tras ROLLBACK.
5	Simulacro: f,f,f,t en autenticación, 1000 filas leídas, ESCALAMIENTO_BLOQUEADO. Tras corrección, admin_datos: SELECT=t; INSERT/UPDATE/DELETE=f sobre auditoría.
6	Dump custom 6.420.169 bytes con hash; restauración de 5 categorías, 20.000 clientes, 50.000 productos, 200.000 pedidos y 400.000 detalles; inválidos 0. RPO estimado 64,871 s y RTO 2,664 s. La tarea programada disparó automáticamente en laboratorio y terminó con código 0.
7	20 pedidos y 40 líneas en MongoDB; IDs de pedidos 20/20 iguales al origen, comparaciones de consultas concordantes.
8	C1–C7: borrador offline, conflicto de precio resuelto con nuevo UUID, rechazos por stock y baja, duplicado bloqueado, CHECK y FK exigidos. Hash y conteos de PostgreSQL iguales antes/después del semáforo. La variante OpenCode coincidió con la versión anterior en ocho solicitudes y tres errores de integridad; el hash de filas SQLite permaneció igual al añadir el índice.

El post incidente de Unidad 6 está en tp8/EVIDENCIA.md y el log de restauración; el de seguridad en tp7/docs/informe_incidente.md. El RPO indicado mide edad del respaldo, no pérdida efectiva de transacciones. Ningún tiempo se extrapola a producción.

IA y decisiones de integración

Codex redactó las specs de Unidades 4 y 8 y coordinó las pruebas. OpenCode Plan propuso diseños y revisó scripts; OpenCode Build generó la variante SQLite adoptada. Los prompts, salidas y decisiones están en tp7/BITACORA.md, tp8/REVISION_OPENCODE.md, tp9/IA_CONTRASTE.md, tp10/docs/bitacora_ia.md y tpi2/revision_final_codex.md. En siete lotes y un suplemento se revisaron los 110 scripts del inventario, con una fila por ruta y decisiones sobre cada hallazgo. Se corrigieron el permiso de auditoría, la guarda FNBC previa al commit, la referencia a la base TP4 y el falso éxito potencial del respaldo; se descartaron hipótesis que no coincidían con la evidencia. Ningún resultado de SQL se atribuye a una ejecución global que no se hizo.

Estado de entrega: la matriz de los 14 objetivos está en tpi2/checklist_entrega.md. Los componentes técnicos están documentados en esta rama, que también conserva PostgreSQL, MongoDB y SQLite. Por solicitud del estudiante se usó Codex donde la guía menciona Kiro; la cátedra debe aceptar esa sustitución. La revisión y sus decisiones están listas para lectura y defensa del estudiante; no se afirma una ratificación personal que no ocurrió.